

Cheatsheet: Ein Quarto-Buch erstellen

Von der Installation bis zum veröffentlichten Buch

ERGEBNIS	AUFBAU	GEEIGNET FÜR
Dokument mit mehreren Kapiteln (HTML / PDF)	Ein Projekt, die Liste <code>chapters:</code> bestimmt die Reihenfolge	Skripte, Handbücher, Abschlussarbeiten

1 – Einrichten

Installieren (einmal je Rechner):

- **VS Code:** code.visualstudio.com (bei der Installation „Add to PATH“ anhängen)
- **Quarto CLI:** quarto.org/docs/get-started
- **Quarto-Extension** für VS Code: in der Seitenleiste *Extensions* nach „Quarto“ suchen

Projekt anlegen: `File` → `New File...` → `Quarto Project` → `Book Project`, dann den Ordner wählen. Quarto legt an:

- `_quarto.yml`: Titel, Autor und Reihenfolge der Kapitel
- `index.qmd`: Titelseite bzw. Vorwort
- `intro.qmd`, `summary.qmd`: Beispielkapitel (umbenennen oder ersetzen)
- `references.bib` / `references.qmd`: Literaturdatei und die Seite, auf der das Verzeichnis erscheint

Was brauche ich?

Du schreibst etwas Langes, das von vorn nach hinten gelesen wird, etwa ein Vorlesungsskript, ein Handbuch oder eine Abschlussarbeit, mit automatischem Inhaltsverzeichnis und Links zum vorigen und nächsten Kapitel? Dann ist das hier richtig. Für lose verlinkte einzelne Seiten nimm das Cheatsheet **Website**, für einen kurzen Handzettel zum Drucken das Cheatsheet **Übungsblatt**.

2 – Kapitel ordnen und schreiben

Jedes Kapitel ist eine `.qmd`-Datei. Die Reihenfolge in `_quarto.yml` ist die Reihenfolge im Buch:

```
book:
  title: "Mein Vorlesungsskript"
  author: "Dein Name"
  chapters:
    - index.qmd
    - intro.qmd
    - summary.qmd
    - references.qmd
```

Ein Kapitel ergänzen: die neue `.qmd`-Datei anlegen und ihren Dateinamen in die Liste `chapters`: eintragen.

Geschrieben wird jedes Kapitel wie jede andere Quarto-Seite, mit `##`-Überschriften, Codezellen, Hinweiskästen und Zitaten. Markdown in Kürze steht im Cheatsheet Website.

Tipp

Leser bekommen eine **Seitenleiste** mit allen Kapiteln, Links zum **vorigen und nächsten** Kapitel am Ende jeder Seite und ein **Inhaltsverzeichnis** des aktuellen Kapitels. Alles entsteht automatisch aus der Liste `chapters`:.

Vorschau und Rendern:

Befehl	Was er tut	Wann
<code>quarto preview</code>	Öffnet eine Vorschau mit Seitenleiste und Navigation, aktualisiert sich beim Speichern	Beim Schreiben und Bearbeiten
<code>quarto render</code>	Baut das fertige Buch nach <code>_book</code> (oder <code>docs/</code> , falls eingestellt)	Vor dem Veröffentlichen

3 – Mit GitHub Pages veröffentlichen

Der Ordner `_book` eines gebauten Buchs funktioniert genau wie der Ordner `_site` einer Website. Die Schritte sind dieselben:

1. In `_quarto.yml` unter `project:` die Zeile `output-dir: docs` ergänzen
2. Rendern: `quarto render`
3. Auf github.com ein **öffentliches** Repository anlegen
4. Den ganzen Projektordner (mit `docs/`) hochladen und committen
5. Im Repository **Settings** → **Pages**: Branch `main`, Ordner `/docs`, dann **Save**
6. Auf den grünen Haken unter **Deployments** warten. Dein Buch steht dann unter `https://<name>.github.io/<repo>/`

Das Buch später aktualisieren: `quarto render` erneut ausführen und den Ordner `docs/` wieder zu GitHub hochladen (oder `git push`, wenn du mit Git arbeitest).

Gut zu wissen

- Kapitel lassen sich mit `part:` in der Liste `chapters:` zu benannten Teilen zusammenfassen, z. B. `part: "Grundlagen"` mit einer eigenen eingerückten Liste `chapters:`.
- Eine Liste `appendices:` auf der Ebene von `book:` ergibt automatisch gezählte Anhänge A, B, ...
- Auf jedes Kapitel lässt sich von überall im Buch verweisen: der Überschrift mit `{#sec-intro}` eine Kennung geben und mit `@sec-intro` darauf verweisen.

Ausführlich und mit Bildschirmfotos: [Was du brauchst](#) · [Buch erstellen](#) · [Website veröffentlichen](#) (dieselben Schritte)